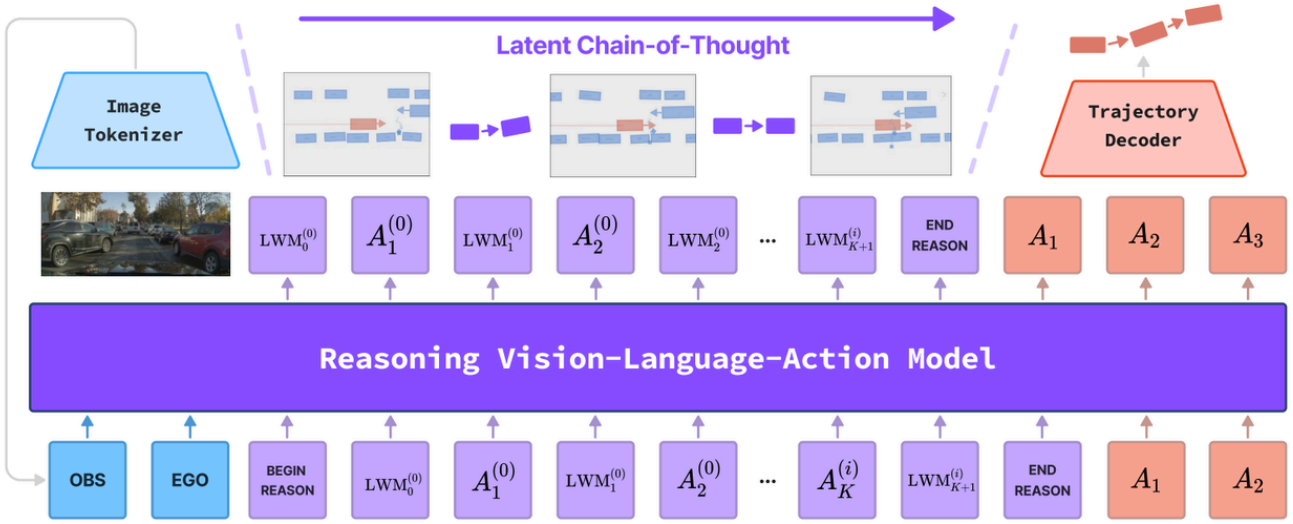
Latent Chain-of-Thought World Modeling for End-to-End Autonomous Driving

> #WorlModel #Finetuning #RL #HybridArchitecture



- What problem does this architecture solve?
  - The current VLAs use natural language text to output CoT's. However, natural language is not the best way to represent special geometry.
  - Using NL has high latencies.
  - The alignment of NL and actual action is weak.
- How does this architecture work?
  - Because of the drawbacks listed above, instead of using pure NL, we use NL action tokens A_t (for actions) and latent tokens LWM_{t+1} (for world states) to populate the context.
  - The context will be like:

    $A_0 \to LWM_1 \to A_1 \to LWM_2 \ldots$, alternatively contains actions and latent tokens.
    - The action is **directly** produced by the model (in NL), and the latent token is produced by the model plus an NLP head. Both tokens will be generated together (from two different places in the same round) and alternatively added to the context.
  - Multi-branch design: To encourage the model to explore different scenarios, the architecture can generate different branches:

    $R^{(i)} = [A_0^{(i)}, LWM_1^{(i)}, A_1^{(i)}, LWM_2^{(i)}, \ldots, A_{K-1}^{(i)}, LWM_K^{(i)}]$ for ith branch, where K is the # of steps.
  - Training:
    - Non-reasoning Pretraining
      - Start with a non-reasoning VLA. SFT to predict the trajectory tokens from the driving data. One copy of the finetuned VLA will be frozen and served as the generator of supervised training data, and the other will serve as the base model.
    - CoT Cold Start

        - Using teacher-forcing + SFT to finetune the model to generate the alternative pattern. Based on the model from the last stage, generate a set of candidates trajectories. Based on the trajectories and the ground truth actions, generate the LWM latent tokens. Insert the LWM latent tokens into the context. Use these to finetune the model. The loss is:

          $\mathcal{L}_{token} = \sum_{i=1}^{B} \sum_{t=0}^{K-1} CE(A_t^{(i)}, \tilde{A}_t^{(i)}) + CE(a_{1:64}, a_{1:64}^*)$ for actions,

          $\mathcal{L}_{lwm} = ||LWM_0 - L\tilde{W}M_0||_2^2 + \sum_{i,t} ||LWM_{t+1}^{(i)} - L\tilde{W}M_{t+1}^{(i)}||_2^2$ for latent tokens,

          $\mathcal{L}_{stage-1} = \mathcal{L}_{token} + \lambda \mathcal{L}_{lwm}$
    - Reinforcement Learning
        - For this step, we use GRPO.
        - Let the reward be:

          $r = -ADE(\hat{\tau}, \tau^*) = -\frac{1}{64} \sum_{i=1}^{64} ||\hat{p}_i - p_i^*||_2$. This calculates the mean Euclidean distance error between the model-predicted final trajectory waypoints ($\hat{p}_i$) and the true expert waypoints (p_i^*).
        - Run GRPO to enforce the model's understanding of the logic.
- Experiment Results
    - Significantly outperforms baselines: In terms of trajectory accuracy (ADE) and safety (fewer road deviations and fewer collisions), LCDrive outperforms both non-inference baseline models and Text-CoT models that utilize massive amounts of text annotations.
    - Extremely Efficient: Unlike natural language, which often generates dozens or even hundreds of redundant descriptive tokens, Latent CoT requires only a few dozen compact vector tokens to achieve high-quality reasoning, resulting in faster inference speeds.
    - Reinforcement Learning Amplification Effect: Experiments demonstrate that the benefits of applying RL (Reinforcement Learning) to Latent CoT far exceed those of applying it to non-reasoning models. This indicates that RL truly “activates” the model’s ability to engage in interactive trial-and-error using the world model.
- Work Cited
    - Paper
    - Chat History
    - NotebookLM